

blog_02

接下来，我们该讲解 何进行流式输出

目录部分

- [blog_02](#)
 - [目录部分](#)
 - [代码部分](#)
 - [代码解析](#)
 - [部分一](#)
 - [部分二](#)
 - [拆开这颗树](#)
 - [回答前面的两个问题](#)

代码部分

```
from openai import OpenAI

client = OpenAI(
    api_key="你的API-KEY",
    base_url="https://api.deepseek.com"
)

# 关键: 设置 stream=True
stream = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "请给我讲个笑话"}
    ],
    stream=True    # 开启流式
)

# 逐块获取生成内容
for chunk in stream:
    # 每个 chunk 是一个 ChatCompletionChunk 对象
    if chunk.choices[0].delta.content:    # 内容可能为 None
        print(chunk.choices[0].delta.content, end="", flush=True)
```

代码解析

部分一

首先第一部分，我们其实没啥好解释的，就是最前面，把response给打开了

```
stream=True
```

但是后面的部分，就不一样了

部分二

```
# 逐块获取生成内容
for chunk in stream:
    # 每个 chunk 是一个 ChatCompletionChunk 对象
    if chunk.choices[0].delta.content:    # 内容可能为 None
        print(chunk.choices[0].delta.content, end="", flush=True)
```

依旧的，这里的`chunk`和`stream`都是我们自己设定的变量，这个名字你咋改都可以，比 这样

```
YAJU = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "你是日本的野兽先辈"},
        {"role": "user", "content": "兄啊，给我讲个关于homo的笑话吧（急不可耐）"}
    ],
    stream=True    # 开启流式
)

# 逐块获取生成内容
for homo in YAJU:
    if homo.choices[0].delta.content:
        print(homo.choices[0].delta.content, end="", flush=True)
```

顺便一提，这是生成的内容

```
Hello, World!
1.3.2
2.38.0
（面无表情地推了推眼镜）哼哼...你这homo小鬼，竟敢跟野兽先辈开这种玩笑。我可是每天都要喝114514升红奈
```

我认为AI在`inm`领域的训练还是少了，需要进一步加强

开个玩笑，请不要到处乱玩`inm`梗，这很不尊重人，也很小鬼，我用这个例子只是想说明，变量的名字并不是重点，防止有人去**傻逼CSDN**上搜索，`openai`库的`chunk`是什么

chunk: a short thick piece or lump (as of wood or coal)

注：按照英文解释，`chunk`就是一块东西，由于流式的时候，是一个一个蹦的，我们便习惯把这里的变量命名为`chunk`，无论你在这里是，命名成`chunk`还是`Chunk`还是`Chankie_Chan`，`Chunk_Sama`还是`HOmo`，`homo`抑或者`i`，理论上都可以

但还是希望你命名成`chunk`，按照习惯

我以前真上CSDN搜索过，——，——

重点在于这里的 `.choice[0].delta.content`

你一定会问，为啥这里会有 `if chunk.choice[0].delta.content:` 的代码，你已经知道中，任何非空字符串的布尔值都是`True`，为啥这里要加一个判断逻辑呢？

还有为啥，这里是 `print(chunk.choices[0].delta.content, end="", flush=True)` 呢？ `end` 是什么？为啥这里要把 `flush` 设置为 `True` 呢？

还是像我的 **blog_01.md** 一样，我要把流式输出的结构树给列出来，让你明白这里的参数

流式输出的结构树

```
chat: Chat
```

```
└─ completions: Completions
```

```
    └─ .create(model, messages, stream=True, ...) → Stream[ChatCompletionChunk]
```

```
    |
    |─ 这是一个"迭代器"对象
    |   # 用 for 循环逐块取，每次吐出
    |
    |─ 每次迭代 → ChatCompletionChunk
    |
    |   └─ id: str
    |       # 这次请求的唯一 ID，
    |
    |   └─ choices: list[Choice]
    |       └─ Choice
    |           └─ index: int
    |               # choices
    |           └─ delta: ChatCompletionDelta
    |               └─ role: str
    |                   # 只在 stream 模式下有效
    |               └─ content: str
    |                   # 只在 stream 模式下有效
    |               └─ reason: str
    |                   # DeepSeek 特有的
    |               └─ finish_reason: str
    |                   # None → 还在生成
    |                   # "stop" → 生成结束
    |
    |   └─ model: str
    |       # 实际使用的模型名
    |
    |   └─ created: int
    |       # Unix 时间戳
    |
    |   └─ usage: CompletionUsage
    |       # 只在最后一个 chunk 有效
    |       # 告诉你总共花了多少 token
```

拆开这颗树

跟 blog_01 里那个一次性返回的 `ChatCompletion` 不一样，`stream=True` 返回的是一个 **迭代器**，不是完整的回复。它不会一次性把 AI 的回复塞给你，而是像拧开的水龙头——拧一点，出一段。

最核心的区别就两处：

1. `delta` 替代了 `message`

还记得 blog_01 里我们取回复用的是 `response.choices[0].message.content` 吗？在流式模式下，`message` 这个字段不存在了，换成了 `delta`。

```
# 非流式 (blog_01)
response.choices[0].message.content

# 流式 (本篇)
chunk.choices[0].delta.content
#                               ↑
#                               这里变了
```

为什么叫 `delta`？因为每个 `chunk` 只携带“增量”——AI 新写出来的那一小段字，而不是从头到尾的全部回复。把所有 `chunk` 的 `delta.content` 拼起来，才是完整的回复。

就好比，你收快递：

- 非流式（`stream=False`）：快递员把所有包裹一趟送齐，你打开箱子，里面是完整的回复。
- 流式（`stream=True`）：快递员分 20 次敲门，每次递给你一张纸条，上面写了一两个字。20 张纸条拼起来才是完整内容。

2. `content` 可能是 `None`，`finish_reason` 从 `None` 变成 `"stop"`

AI 在流式输出的时候，大部分 `chunk` 都带着文字（`content` 有值，`finish_reason` 为 `None` ——意思是“还在写呢别急”）。但最后一个 `chunk` 不一样：它的 `content` 是 `None`，`finish_reason` 变成 `"stop"`，意思是“我写完了，没东西了”。

这就是为啥代码里要加 `if` 判断，也是接下来要回答的核心问题。

回答前面的两个问题

第 109 行的问题：为啥这里会有 `if chunk.choices[0].delta.content:` 的判断？你已经知道任何非空字符串的布尔值都是 `True`，为啥还要加一个判断逻辑呢？

因为 `delta.content` 不总是字符串——它可能是 `None`。

流式输出的最后一个 chunk 长这样（伪代码）：

```
# 最后一个 chunk 大概长这样
ChatCompletionChunk(
    id="xxx",
    choices=[
        Choice(
            index=0,
            delta=ChatCompletionDelta(
                content=None,          # ← 空的!
                role=None
            ),
            finish_reason="stop"      # ← AI 说：我说完了，收工
        )
    ],
    usage=CompletionUsage(...)      # usage 也只有最后一个 chunk 有
)
```

果你不写 `if chunk.choices[0].delta.content:`，代码会直接执行 `print(None, end="", flush=True)`，屏幕上就会凭空多出四个大字 `None`，把好好的回复给污染了。

所以这里的 `if` 不是在判断“字符串是不是空串”，而是在判断“这个 chunk 到底带没带文字”。带了 → 打印。没带（最后一个 chunk）→ 跳过。

第 111 行的问

题： `print(chunk.choices[0].delta.content, end="", flush=True)` —— `end` 是什么？为啥要把 `flush` 设为 `True`？

`end=""`

Python 的 `print()` 默认会在输出完内容之后自动加一个换行符（`\n`）。 果不覆盖它：

```
# 默认行为（end 默认是 "\n"）
print("你")
print("好")
# 输出：
# 你
# 好
```

你每一个 chunk 都占一行，全给你换行了，根本没法看。流式输出要求每个 chunk **紧挨着上一个**，像正常打字一样连贯。所以 `end=""` 就是告诉 `print`：别加换行，下一个 chunk 直接贴在我屁股后面。

```
# end=""
print("你", end="")
print("好", end="")
# 输出：
# 你好
```

flush=True

Python 的输出有个"缓冲区"机制——为了性能，字符会先攒在内存里，攒够一批再一次性刷到屏幕上。对于普通程序这完全没问题，但对于流式输出：

- 你 果不 `flush`，用户可能盯着空白的终端等 5 秒，然后"啪"一下刷出一大段字 —— 跟"打字机效果"没有任何关系。
- `flush=True` 就是在说：**别攒了，有一个字就给我立刻显示一个字。**

`flush=False`（默认行为）：

[攒...攒...攒...哗！] → 一口气全吐出来

`flush=True`：

你 → 好 → 啊 → ， → 我 → 是 → A → I （逐字逐字往外蹦）

这两个参数凑在一起——`end=""` 让字不换行，`flush=True` 让字不缓冲——才实现了那种"AI 一边想一边往外蹦字"的打字机效果。